

講評と 設計の説明

コードの振る舞いを、根拠とともに話す

情報システム学科 3年 / 選択2単位

今日の到達目標

- 実装の結果を仕様と照合し、改善点を説明できる
- 小さい反例で、探索法の保証の違いを示せる
- 自分の提出物について、設計と検証の質疑に答えられる

共通講評には教材用の架空提出例を使う。実際の学生の情報は含めない。

前回との接続

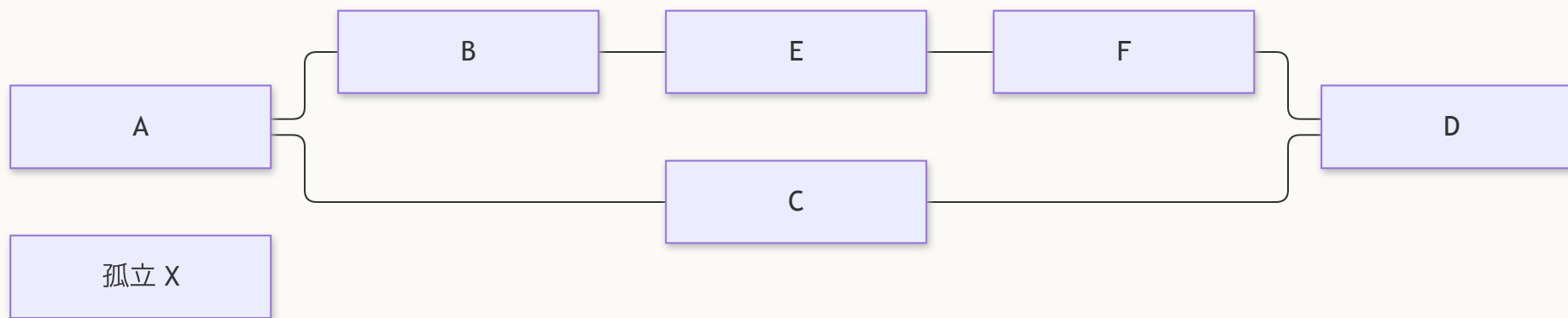
前回の成果物は、コードと検証記録と振り返り。

今日は「なぜその結果になるか」をコードに戻って確認する。

- 仕様の言葉と戻り値が一致するか
- テストは、その仕様を確認できる入力か
- AIの提案を採用した理由を説明できるか

講評の対象

この回のサンプルは **講評専用の架空グラフ**。第06回の駅データとは異なる。



AからDには、4辺と2辺の経路がある。

架空提出例の主張

Python 3.12以上の環境で実行する。

`flawed_route.py` は、DFSで最初に発見した経路を「最短」と表示する。

```
python3 examples/07-review/flawed_route.py A D
```

出力経路は **A B E F D**、移動数は **4**。

経路は存在する。しかし、仕様の「最少辺数」を満たすか確認が必要。

反例の役割

同じ入力に、もっと短い経路 **A C D** がある。移動数は2。

「すべての入力で最少」という主張は、この1例で崩れる。

- 経路の妥当性：隣接頂点が辺で結ばれる
- 最少性：これより短い経路がない

成立している部分と、改善が必要な部分を分けて講評する。

DFSの処理順

Aの隣接順がB、Cなら、B側の奥を先にたどる。

進行	現在の経路
Aを訪問	A
B側へ進む	A B
E、Fへ進む	A B E F
Dを発見	A B E F D

訪問済みの管理は、探索の終了に役立つ。最少性の保証とは別の条件。

BFSの処理順

BFSは、出発点からの辺数が小さい順に探索する。

層	頂点
0	A
1	B, C
2	E, D

Dの発見に必要な辺数は2。先にBを処理しても、B側の奥まで進まない。

改善版の照合

```
python3 examples/07-review/improved_route.py A D
```

期待する経路は **A C D**、移動数は **2**。

改善点：FIFOのキューと、最初に発見した親の記録を使う。

コードを比較し、何が違って最少性を保証するかを説明する。

仕様に戻る説明

説明の出発点は、関数名よりも要求する答え。

「無向・重みなしのグラフで、最少辺数の経路1本を返す」

この条件に対して、BFSは近い層を先に探索する。最初の親を保持すると、その層の経路を復元できる。

重みが異なる場合には、この根拠をそのまま使えない。

入力変更による確認

講評用グラフで、A Cの辺を削るとどうなるか。

残る経路は **A B E F D**、移動数は4。

- 変更前：4辺を返す実装は最少性を満たさない
- 変更後：4辺の経路も最少になる

1つの成功例だけで、実装一般の正しさを判断しない。

境界ケースの講評

条件	確認する振る舞い
同じ出発点と到着点	経路1頂点、移動数0
孤立したXへの探索	到達不能として終了
未知の頂点名	入力エラーとして終了
循環を含むグラフ	重複処理で終わらなくなる

未対応の条件には、再現入力と期待値を添える。

テストの不足

架空提出例が「Aから隣のB」だけを確認していたとする。

その入力では、DFSでもBFSでも1辺の経路を返す。

テストを増やすときは、数よりも **区別できる条件** を選ぶ。

この例では、奥の4辺と近い2辺を持つ入力、最少性の違いを明らかにする。

計算量の説明

隣接リストのBFSでは、各頂点と隣接要素を一通り調べる。

- 時間 $O(V + E)$ 、追加領域 $O(V)$
- 途中で全経路をコピーするなら、その費用も検討する
- 出力する経路の長さも、復元の処理に影響する

「速い」という感想を、入力サイズと操作数の説明へ直す。

読みやすさの講評

名前は、記録する意味と対応させる。

```
parent[neighbor] = node
```

この1行は「neighborを最初に発見した直前の頂点をnodeと記録する」。

変数名・関数の責務・境界処理が追えると、誤りを説明しやすい。

講評文の書き方

対象を特定し、根拠を示し、改善後の確認方法を書く。

例：

A Dで4辺の経路を返しますが、A C Dは2辺です。FIFOのキューで探索し、同じ入力と到達不能の入力で結果を確認してください。

人格や能力の評価を避け、修正できる振る舞いを扱う。

演習1：架空提出例のレビュー

`exercises/07.md` に沿って、両方の実装を実行する。

- 主張、反例、実際の出力を記録する
- 仕様を満たす点と、改善が必要な点を書く
- 改善後の確認入力を1つ追加する

成果物：`07-review.md`。

演習2：自分の提出物の説明

第06回のコードを開き、次の箇所を指して説明する。

1. 仕様と、入力エラーの検証
2. キューの初期化と取り出し
3. 発見済みの判定と親の記録
4. 到達不能の判定と経路復元

コードを読むだけでなく、具体的な入力で状態を追う。

授業内の質疑

提出者と担当教員で、提出内容の理解を確認する。

問いの例：「AからDの探索で、Cを処理した時点の親記録を説明してください」

答えに詰まったら、入力を小さくし、キューと辞書を書いて確かめる。

説明の修正や残る疑問を、その場でメモへ残す。

待機中はペアで本人コードを説明し、反例を検証する。教員との確認順は授業で案内する。

AI利用の振り返り

AIが生成した箇所も、採用した実装について説明する。

- どの問いにAIを使ったか
- 提案のどこを変更したか
- 何を実行して正しさを確認したか
- 今なら、自分でどこまで説明できるか

AI利用の有無だけで、理解の有無を決めない。

改善記録

07-improvements.md に、修正前後の差を残す。

- 直した仕様・振る舞い
- 再現入力、期待値、実際の結果
- 修正の理由
- 次に確認したい条件

正式な再提出の有無や方法は、授業時の案内に従う。

確認問題

1. 訪問済み集合があれば、DFSは最短経路を保証するか
2. 経路が存在することと、最短であることはどう違うか
3. どんなテスト入力なら、DFSとBFSの違いを検出しやすいか
4. 修正後のコードを、どの証拠で確認するか

理由と具体例を添えて答える。

まとめと次回

講評では、仕様・反例・状態の追跡を結び付ける。説明できない部分は、小さい入力で再検証する。

次回は、電卓を題材に **字句解析・構文解析・意味解析** を学ぶ。

入力文字列を木構造へ変換する場面でも、今日のデータ構造の考え方をを使う。

参考資料

- [Python公式：collections.deque](#)
- [MIT 6.006：BFS・DFSの講義資料](#)

架空提出例・グラフ・講評文は本教材用の自作。実学生の提出物や採点情報は使っていない。